

8086 Arithmetic Instructions

1. Quick Overview

Concept

Arithmetic instructions perform math on 8-bit or 16-bit data. In 8086, some operations use **implicit registers** (like AL, AX, DX:AX), so always write where the input comes from and where the result goes.

2. INC and DEC

Concept

INC increases an operand by 1. **DEC** decreases an operand by 1.

Rules / Notes

Syntax

```
INC destination  
DEC destination
```

Notes

- Operand can be register or memory
- Works for 8-bit and 16-bit operands

Example

```
INC CX  
DEC DL  
INC [COUNT]
```

Example

```
; Direct memory (actual address via offset)
INC [2000H]      ; increment the byte/word stored at DS:2000H
DEC [2002H]      ; decrement the byte/word stored at DS:2002H
```

Explanation: This uses **Direct Memory Addressing**. The offset is written explicitly. The CPU accesses the memory location using the Data Segment (DS) by default:

$$\text{Physical Address} = DS \times 10H + \text{Offset}$$

So for `INC [2000H]`, the CPU increments the value stored at

$$DS : 2000H$$

(e.g., if the value was `3FH`, after `INC` it becomes `40H`).

Tips

Common exam point: **INC/DEC do not affect Carry Flag (CF)**.

3. ADD and SUB

Concept

ADD performs binary addition. **SUB** performs binary subtraction.

Rules / Notes

Syntax

```
ADD destination, source
SUB destination, source
```

Rules

- register $\leftrightarrow$ register is allowed
- register $\leftrightarrow$ memory is allowed
- immediate with register/memory is allowed
- **memory-to-memory is not allowed**

Example

```
MOV AX, [NUM1]
MOV BX, [NUM2]
ADD AX, BX
```

Explanation: Load operands then add. Result stored in AX.

Example

```
; ----- Direct memory addressing -----
MOV AX, [2000H]      ; AX <- word at DS:2000H
ADD AX, 0005H        ; AX = AX + 5
MOV [2000H], AX      ; store result back to memory
```

Explanation: The value stored at offset 2000H is first loaded into AX, then incremented, and finally written back. Direct memory-to-memory addition is avoided by using AX as an intermediate register.

Example

```
; ----- Register indirect / based addressing -----
MOV BX, 1000H
ADD AX, [BX]          ; AX = AX + value at DS:1000H

SUB AX, [BX + 04H]    ; AX = AX - value at DS:(BX+04H)
```

Explanation: For ADD AX, [BX], the effective address is:

$$EA = BX$$

For SUB AX, [BX + 04H], the effective address is:

$$EA = BX + 04H$$

The Data Segment (DS) is used by default for address formation.

Tips

Why memory-to-memory ADD/SUB is not allowed: 8086 ALU operates on registers. At least one operand must be in a register, so memory values must be loaded into a register before arithmetic.

4. ADC (Add with Carry)

Concept

ADC adds the source, destination, and the Carry Flag (CF): it is used for multi-byte (multi-precision) addition.

Rules / Notes

Syntax

```
ADC destination, source
```

Operation

$$destination \leftarrow destination + source + CF$$

Rules

- Destination: register or memory
- Source: immediate / register / memory
- Two memory operands are not allowed

Example

```
; Add 16-bit low parts, then high parts using carry
ADD AX, BX          ; low word addition
ADC DX, CX          ; high word + carry
```

Example

```
; Add two 32-bit numbers stored in memory
; NUM1 = [2000H:2002H], NUM2 = [3000H:3002H]

MOV AX, [2000H]      ; low word of NUM1
ADD AX, [3000H]      ; add low word of NUM2
MOV [4000H], AX      ; store low word of result

MOV AX, [2002H]      ; high word of NUM1
ADC AX, [3002H]      ; add high word + carry
MOV [4002H], AX      ; store high word of result
```

Explanation: The lower 16 bits are added using **ADD**. If a carry occurs, it is stored in CF and automatically added to the higher 16 bits using **ADC**. This is the correct method for multi-precision arithmetic in 8086.

Example

```
; Using register indirect addressing
MOV SI, 2000H
MOV DI, 3000H

MOV AX, [SI]          ; low word of first number
ADD AX, [DI]          ; add low word
MOV [SI], AX          ; store result

MOV AX, [SI + 2]      ; high word of first number
ADC AX, [DI + 2]      ; add high word + carry
MOV [SI + 2], AX
```

Explanation: Effective addresses are computed as:

$$EA_{\text{low}} = SI, \quad EA_{\text{high}} = SI + 2$$

ADC ensures that any carry from the lower addition is correctly propagated to the higher word.

Tips

ADC does not distinguish signed/unsigned. CPU updates flags so that **CF** indicates unsigned carry, **OF** indicates signed overflow.

5. SBB (Subtract with Borrow)

Concept

SBB subtracts source and the Carry Flag (CF) from destination. It is used for multi-precision subtraction (borrow chaining).

Rules / Notes

Syntax

```
SBB destination, source
```

Operation

$$destination \leftarrow destination - source - CF$$

Rules: same operand restrictions as SUB (no memory-to-memory).

Example

```
; Multi-word subtraction (low then high using borrow)
SUB AX, BX
SBB DX, CX
```

Example

```
; Subtract two 32-bit numbers stored in memory
; NUM1 = [2000H:2002H], NUM2 = [3000H:3002H]
; RESULT will be stored at [4000H:4002H]

MOV AX, [2000H]      ; low word of NUM1
SUB AX, [3000H]      ; subtract low word of NUM2
MOV [4000H], AX      ; store low word result

MOV AX, [2002H]      ; high word of NUM1
SBB AX, [3002H]      ; subtract high word + borrow (CF)
MOV [4002H], AX      ; store high word result
```

Explanation: First, **SUB** subtracts the low words. If a borrow occurs, the CPU sets **CF=1**. Then **SBB** subtracts the high words and also subtracts this borrow automatically.

Example

```
; Register-indirect subtraction of 32-bit numbers
MOV SI, 2000H
MOV DI, 3000H

MOV AX, [SI]         ; low word of first number
SUB AX, [DI]         ; subtract low word
MOV [SI], AX         ; store result low word

MOV AX, [SI + 2]     ; high word of first number
SBB AX, [DI + 2]     ; subtract high word + borrow
MOV [SI + 2], AX
```

Explanation: Effective addresses:

$$EA_{\text{low}} = SI, \quad EA_{\text{high}} = SI + 2$$

Borrow from the low subtraction is carried in CF and applied using SBB.

Example

```
; Demonstrating borrow propagation
MOV AX, 0000H
SUB AX, 0001H      ; AX = FFFFH, CF = 1 (borrow occurred)
MOV DX, 0002H
SBB DX, 0000H      ; DX = 0001H (because CF=1 is subtracted)
```

Tips

Rule for multi-word subtraction: Use **SUB** for the least-significant word, then use **SBB** for all higher words to include the borrow stored in CF.

6. MUL and IMUL

Concept

MUL multiplies **unsigned** numbers. **IMUL** multiplies **signed** numbers. Both affect **CF** and **OF**.

Rules / Notes

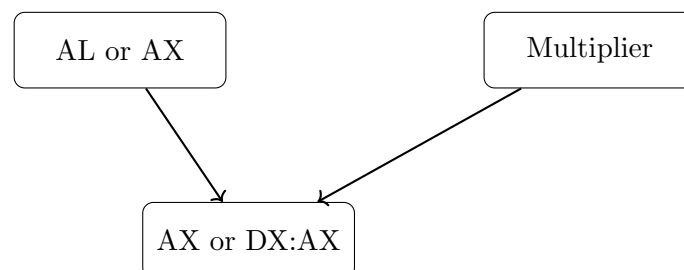
Syntax

```
MUL  r/m8    or    MUL  r/m16
IMUL r/m8    or    IMUL r/m16
```

Result placement

- 8-bit: $AL \times \text{operand} \rightarrow AX$
- 16-bit: $AX \times \text{operand} \rightarrow DX:AX$

Result Placement Diagram



Example

```
MOV AL, 10
MOV DL, 25
MUL DL      ; AX = AL * DL
```

Example

```
; 8-bit unsigned multiplication with memory operand
MOV AL, 12H          ; AL = 18 (decimal)
MUL BYTE PTR [2000H]
```

Explanation: The byte at memory location DS:2000H is multiplied with AL.

$$AX = AL \times [DS : 2000H]$$

The 16-bit product is stored in AX. If AH $\neq$ 0, then CF and OF are set.

Example

```
; 16-bit unsigned multiplication
MOV AX, 0100H
MUL WORD PTR [3000H]
```

Explanation: AX is multiplied with the 16-bit value at DS:3000H.

$$DX : AX = AX \times [DS : 3000H]$$

The lower 16 bits go into AX, and the upper 16 bits go into DX.

Example

```
; Using register indirect addressing
MOV SI, 2000H
MOV AX, 0020H
MUL WORD PTR [SI]
```

Explanation: The effective address is:

$$EA = SI$$

The value at DS:SI is multiplied with AX. The 32-bit result is stored in DX:AX.

Example

```
; Signed multiplication
MOV AL, 0F6H      ; -10 (signed)
MOV BL, 05H       ; +5
IMUL BL
```

Explanation: This is signed multiplication:

$$-10 \times 5 = -50$$

The signed 16-bit result is stored in AX (two's complement form). CF and OF are set if sign-extension is required.

Example

```
; Signed 16-bit multiplication with memory
MOV AX, 0FF9CH    ; -100 (signed)
IMUL WORD PTR [4000H]
```

Explanation: AX is multiplied with the signed 16-bit value at DS:4000H. The signed 32-bit result is stored in DX:AX.

Tips

MUL/IMUL always use **AL or AX implicitly**. You never write the destination operand explicitly. For 16-bit multiplication, always mention **DX:AX** as the result. CF and OF are set if the upper half of the product is non-zero (or sign-extension is needed for IMUL).

7. DIV and IDIV

Concept

DIV divides **unsigned** data, **IDIV** divides **signed** data. Dividend is in implicit registers; quotient and remainder go to implicit registers.

Rules / Notes

Syntax

```
DIV divisor
IDIV divisor
```

Case 1: divisor is 8-bit (r/m8)

- Dividend: AX (16-bit)
- Quotient: AL (8-bit)
- Remainder: AH (8-bit)

Case 2: divisor is 16-bit (r/m16)

- Dividend: DX:AX (32-bit)
- Quotient: AX (16-bit)
- Remainder: DX (16-bit)

Tips

Note on Status Flags (Important):

According to standard Intel documentation, the status flags after DIV and IDIV are **undefined** and should not be relied upon.

However, **this lecture specifies** that DIV/IDIV affect all six status flags (OF, SF, ZF, AF, PF, CF).

Exam Rule: Always follow the definition given by the instructor or lecture notes.

Example

```
; Divide 80 by 2
MOV AX, 80
MOV BL, 02
DIV BL
; AL = AX / BL    (quotient)
; AH = AX % BL    (remainder)
```

Example

```
; 8-bit divisor stored in memory (actual address)
; AX is dividend, divisor is byte at DS:2000H
MOV AX, 1234H
DIV BYTE PTR [2000H]
; Quotient -> AL, Remainder -> AH
```

Explanation: Divisor is [DS:2000H] (8-bit), dividend is AX (16-bit). After DIV:

$$AL = \frac{AX}{\text{divisor}}, \quad AH = AX \bmod \text{divisor}$$

Example

```
; 16-bit divisor in memory, 32-bit dividend in DX:AX
MOV DX, 0001H      ; high word of dividend
MOV AX, 0000H      ; low word of dividend -> dividend = 0001:0000H
                    = 65536
DIV WORD PTR [3000H]
; Quotient -> AX, Remainder -> DX
```

Explanation: When divisor is 16-bit, dividend must be 32-bit in DX:AX:

$$AX = \frac{DX : AX}{\text{divisor}}, \quad DX = (DX : AX) \bmod \text{divisor}$$

Example

```
; Register indirect addressing for divisor
MOV SI, 2000H
MOV AX, 0100H
DIV BYTE PTR [SI]   ; divisor is at DS:SI
```

Explanation: Effective address:

$$EA = SI$$

Divisor is fetched from DS:SI. Quotient goes to AL, remainder to AH.

Example

```
; Signed division example (IDIV with 8-bit divisor)
MOV AX, 0FF9CH      ; AX = -100 (signed)
MOV BL, 05H         ; divisor = +5
IDIV BL
; AL = -20, AH = 0   (signed remainder)
```

Explanation: IDIV treats operands as signed numbers. Quotient is stored in AL, remainder in AH.

Example

```
; Signed 16-bit division (IDIV with 16-bit divisor)
MOV DX, 0FFFFH      ; sign-extension high word (for negative
                     ; dividend)
MOV AX, 0FF9CH       ; -100 (signed) represented in DX:AX
IDIV WORD PTR [4000H]
; AX = quotient, DX = remainder
```

Explanation: For 16-bit signed division, the dividend must be in DX:AX (sign-extended). After IDIV:

$AX = \text{quotient}, \quad DX = \text{remainder}$

Tips

Critical warnings:

- Division by zero causes a CPU error (interrupt).
- If the quotient does not fit in AL (for r/m8) or AX (for r/m16), a **divide overflow** occurs.

8. NEG (Two's Complement)

Concept

NEG replaces the operand with its **two's complement**. It is equivalent to: $0 - \text{operand}$.

Rules / Notes

Syntax

```
NEG destination
```

Destination can be register or memory.

Example

```
MOV AL, 05H
NEG AL      ; AL becomes FBH (two's complement of 05H)
```

Example

```
MOV AX, 1234H  
NEG AX
```

Explanation: NEG computes the two's complement:

$$AX = 0 - 1234H = EDCCH$$

This is equivalent to inverting all bits and adding 1.

Example

```
; Direct memory operand  
NEG BYTE PTR [2000H]  
NEG WORD PTR [2002H]
```

Explanation: The value stored at memory location DS:2000H (byte) or DS:2002H (word) is replaced by its two's complement. Physical address used:

$$PA = DS \times 10H + \text{offset}$$

Example

```
MOV SI, 3000H  
NEG WORD PTR [SI]
```

Explanation: Effective address is:

$$EA = SI$$

The word at DS:SI is negated in memory (no register is required).

Example

```
MOV AL, 00H  
NEG AL          ; AL = 00H, CF = 0  
  
MOV AL, 05H  
NEG AL          ; AL = FBH, CF = 1
```

Explanation:

- If the operand is **zero**, CF is cleared (CF = 0)
- If the operand is **non-zero**, CF is set (CF = 1)

Example

```
MOV AL, 80H
NEG AL          ; AL = 80H, OF = 1 (signed overflow)
```

Tips

NEG performs $0 - \text{operand}$. CF = 1 if operand $\neq 0$, otherwise CF = 0. NEG works on both registers and memory.

Signed Overflow Case (Important):

When NEG is applied to the most negative signed number:

- 8-bit: 80H
- 16-bit: 8000H

The result cannot be represented in signed form.

Result:

- Operand remains unchanged
- **Overflow Flag (OF) is set**

9. BCD & ASCII Adjust Instructions

Concept

These instructions correct results after adding/subtracting **BCD or ASCII-coded decimal digits**. They are used immediately after ADD/SUB.

Concept

Packed vs Unpacked BCD (Key Definition):

- **Packed BCD:** Two decimal digits are stored in one byte (upper nibble = tens digit, lower nibble = units digit). Example: decimal 45 $\rightarrow$ 45H
- **Unpacked BCD / ASCII:** Each decimal digit is stored in a separate byte. Example:
 - Unpacked BCD: 04H, 05H
 - ASCII: 34H ('4'), 35H ('5')

Instruction Mapping:

- **AAA, AAS** $\rightarrow$ Unpacked BCD / ASCII
- **DAA, DAS** $\rightarrow$ Packed BCD

9.1 AAA (ASCII Adjust After Addition)

Concept

AAA adjusts the result in AL after adding two **unpacked BCD/ASCII digits**. It produces a valid 1-digit unpacked BCD result in AL and updates AH if needed.

Rules / Notes

When to use: typically after

- clear AH (AH = 00H)
- ADD two unpacked ASCII/BCD digits in AL
- then execute AAA

Adjustment rule (lecture idea):

- If lower nibble of AL ≤ 9 and AF=0: no adjustment, upper nibble becomes 0
- If lower nibble > 9 or AF=1: add 06H to AL (lower nibble adjust), increment AH by 1, set AF and CF to 1

Example

```
sub AH, AH      ; AH = 00
mov AL, '6'     ; AL = 36H
add AL, '7'     ; AL = 6DH
aaa             ; adjust after addition
or  AX, 3030H   ; convert unpacked BCD digits to ASCII codes
```

Step-by-Step Explanation:

- AL = 36H ('6'), add 37H ('7') $\rightarrow$ AL = 6DH
- Lower nibble = DH (13 decimal) > 9
- AAA adds 06H $\rightarrow$ AL = 73H
- AH is incremented by 1 $\rightarrow$ AH = 01H
- Upper nibble of AL is cleared $\rightarrow$ AL = 03H

After OR AX, 3030H:

$AX = 3133H$ (ASCII characters '1' and '3')

Example

```
sub AH, AH
mov AL, '8'
add AL, '9'      ; AL = 71H
aaa
or  AX, 3030H
```

Explanation: $8 + 9 = 17 \rightarrow$ unpacked BCD result. AH contains '1' and AL contains '7'.

9.2 DAA (Decimal Adjust Accumulator)

Concept

DAA adjusts AL after adding two **packed BCD** numbers to form a valid packed BCD result.

Rules / Notes

Adjustment rules (lecture):

- If lower nibble > 9 or $AF=1$, then $AL \leftarrow AL + 06H$
- If upper nibble > 9 or $CF=1$, then $AL \leftarrow AL + 60H$

Example

```
MOV CL, 29H
MOV AL, 53H
ADD AL, CL      ; AL = 7CH
DAA             ; AL = 82H (C > 9 so +06H)
```

Step-by-Step Explanation:

- $53H + 29H = 7CH$
- Lower nibble = CH (>9) $\rightarrow$ add 06H $\rightarrow AL = 82H$
- Upper nibble is valid $\rightarrow$ no +60H

Final packed BCD result:

$$AL = 82H \quad (82 \text{ decimal})$$

Example

```
MOV CL, 29H
MOV AL, 73H
ADD AL, CL      ; AL = 9CH
DAA             ; adjust -> CF may be set after +60H
```

Example

```
MOV AL, 58H
ADD AL, 75H      ; AL = CDH
DAA
```

Explanation:

- Lower nibble $> 9 \rightarrow +06H$
- Upper nibble $> 9 \rightarrow +60H$
- CF is set due to decimal carry

9.3 AAS (ASCII Adjust After Subtraction)

Concept

AAS adjusts AL after subtracting two unpacked ASCII/BCD digits, producing a valid 1-digit unpacked BCD result.

Rules / Notes

Lecture rules:

- If lower nibble > 9 or $AF=1$: $AL \leftarrow AL - 06H$, $AH \leftarrow AH - 1$, set AF and CF
- Otherwise: $AF=CF=0$ and upper nibble of AL becomes 0

Example

```
sub AH, AH
mov AL, '9'      ; 39H
sub AL, '3'      ; 06H
aas
or AL, 30H       ; make ASCII
```

Step-by-Step Explanation:

- $'9' - '3' = 06H$
- Lower nibble ≤ 9 and $AF = 0 \rightarrow$ no adjustment
- Upper nibble cleared $\rightarrow$ valid unpacked BCD

After `OR AL, 30H`, AL becomes ASCII '6'.

Example

```
sub AH, AH
mov AL, '3'
sub AL, '9'          ; FAH (negative style result)
aas                  ; AX becomes FF04H (lecture idea)
```

Example

```
sub AH, AH
mov AL, '2'
sub AL, '8'          ; borrow occurs
aas
or  AX, 3030H
```

Explanation: $2 - 8 = -6 \rightarrow$ result represented using borrow. AH is decremented and AL holds corrected unpacked BCD.

9.4 DAS (Decimal Adjust After Subtraction)

Concept

DAS adjusts AL after subtracting two **packed BCD** numbers to obtain a valid packed BCD result.

Rules / Notes

Lecture rules:

- If lower nibble > 9 then subtract 06H from AL
- If CF=1 (borrow) or upper nibble > 9 then subtract 60H from AL
- DAS modifies CF, AF, PF, SF, ZF; OF is undefined

Example

```
MOV AL, 75H
MOV BH, 46H
SUB AL, BH      ; AL = 2FH, AF = 1
DAS             ; AL = 29H
```

Step-by-Step Explanation:

- $75H - 46H = 2FH$
- Lower nibble = FH ($\downarrow 9$) $\rightarrow$ subtract $06H \rightarrow AL = 29H$
- No upper nibble borrow $\rightarrow$ no $-60H$

Final packed BCD result:

$AL = 29H$

Example

```
MOV AL, 38H
MOV CH, 61H
SUB AL, CH      ; AL = D7H, CF = 1
DAS             ; AL = 77H, CF = 1
```

Example

```
MOV AL, 42H
SUB AL, 78H      ; AL = CAH, CF = 1
DAS
```

Explanation:

- Lower nibble $\downarrow 9 \rightarrow -06H$
- $CF = 1 \rightarrow -60H$
- Result corrected to valid packed BCD

10. AAM, AAD, CBW, and CWD (Adjust & Sign-Extension)

10.1 AAM (ASCII Adjust After Multiplication)

Concept

AAM adjusts the result of multiplying two **unpacked BCD digits**. It converts the binary product in **AL** into two unpacked BCD digits: **AH** = tens digit and **AL** = ones digit (base 10 by default).

Typical use: MUL (product in AX) then AAM (convert to unpacked BCD in AH:AL).

Rules / Notes

Implicit operand/register: **AX** is both source and destination.

Operation (default base = 10):

$$AH \leftarrow \left\lfloor \frac{AL}{10} \right\rfloor, \quad AL \leftarrow (AL \bmod 10)$$

Notes

- Use AAM **after** multiplying unpacked BCD digits (0–9).
- Multiplication should not be performed directly on ASCII codes.
- Status flags: **SF**, **ZF**, **PF** are affected; **CF**, **AF**, **OF** are undefined (course-dependent).

Example

```
; Example 1: multiply unpacked BCD digits (3 x 9 = 27)
MOV AL, 03H          ; multiplier (unpacked BCD digit)
MOV BL, 09H          ; multiplicand (unpacked BCD digit)
MUL BL               ; AX = 001BH (27 decimal)
AAM                  ; AH=02, AL=07 -> AX = 0207H
OR  AX, 3030H        ; convert unpacked BCD to ASCII -> AX = 3237H
                     ('27')
```

Explanation:

- After MUL, AL holds 27 (1BH).
- AAM splits 27 into tens and ones: $AH = \lfloor 27/10 \rfloor = 2$, $AL = 27 \bmod 10 = 7$.
- OR with 3030H converts digits 02 and 07 to ASCII '2' and '7'.

Example

```
; Example 2: digits given as ASCII, convert to unpacked BCD first
MOV AL, '3'          ; 33H
MOV BL, '9'          ; 39H
AND AL, 0FH          ; -> 03H
AND BL, 0FH          ; -> 09H
MUL BL               ; AX = 001BH (27 decimal)
AAM                  ; AX = 0207H
OR  AX, 3030H        ; AX = 3237H ('27')
```

Tips

Key exam line: AAM converts a binary product in AL into two unpacked decimal digits in AH (tens) and AL (ones), assuming base 10.

10.2 AAD (ASCII Adjust Before Division)

Concept

AAD is used **before division** when a two-digit unpacked BCD number is stored in **AH:AL**. It converts these two unpacked digits into a single binary number in **AL**, and clears **AH**.

Rules / Notes

Operation (default base = 10):

$$AL \leftarrow (AH \times 0Ah) + AL, \quad AH \leftarrow 00H$$

Flags: **SF**, **ZF**, **PF** are modified. **CF**, **AF**, **OF** are undefined.

Why we use it:

- **DIV** expects a binary dividend in **AX** (for 8-bit divisor).
- If the dividend is stored as unpacked BCD digits (tens in **AH**, ones in **AL**), **AAD** converts it into binary so division works correctly.

Example

```
; Example: divide 27 by 5 (unpacked BCD dividend)
MOV AX, 0207H      ; AH=02, AL=07 -> "27" in unpacked BCD
MOV BL, 05H        ; divisor (unpacked digit / binary 5)
AAD                ; AX = 001BH (AL=27, AH=0)
DIV BL             ; AL=05 (quotient), AH=02 (remainder) -> 27/5 = 5
                   r 2
```

Explanation:

- **AAD** makes $AL = (2 \times 10) + 7 = 27$, $AH = 0$.
- **DIV BL** then divides $AX (=27)$ by 5: quotient in **AL**, remainder in **AH**.

Example

```
; Given: AX = 0508H (unpacked BCD "58"), divisor = 2
MOV AX, 0508H
MOV DH, 02H
AAD                ; AX = 003AH (AL=58 decimal = 3AH, AH=00)
DIV DH             ; AL = 1DH (29), AH = 00 (remainder)
```

Tips

Key exam line: **AAD** converts two unpacked decimal digits in **AH:AL** into a binary number in **AL** (**AH** cleared), so division can be performed correctly.

10.3 CBW (Convert Byte to Word)

Concept

CBW converts a signed 8-bit value in **AL** to a signed 16-bit value in **AX** by **sign-extending** AL into AH.

Rules / Notes

Operation:

$$AH \leftarrow \begin{cases} 00H, & \text{if } AL \text{ is positive (sign bit 0)} \\ FFH, & \text{if } AL \text{ is negative (sign bit 1)} \end{cases}$$

Notes:

- Preserves the signed value (two's complement).
- Does not affect any flag (as per lecture).
- Often used before **signed** arithmetic on a byte value.

Example

```
; Example: AL = -101 decimal = 9BH (two's complement)
MOV AL, 9BH
CBW                ; AX = FF9BH
```

Explanation: Since AL has sign bit 1, CBW fills AH with FFH, producing AX = FF9BH.

10.4 CWD (Convert Word to Doubleword)

Concept

CWD sign-extends a signed 16-bit value in **AX** into a signed 32-bit value in **DX:AX**. This is commonly done before **signed division** using IDIV.

Rules / Notes

Operation:

$$DX \leftarrow \begin{cases} 0000H, & \text{if } AX \text{ is positive} \\ FFFFH, & \text{if } AX \text{ is negative} \end{cases}$$

Notes:

- Does not affect any flag (as per lecture).
- Used before IDIV **r/m16** because signed dividend must be in **DX:AX**.

Example

```
; Example: AX = -101 decimal = FF9BH
MOV AX, FF9BH
CWD                ; DX:AX = FFFFH:FF9BH
```

Explanation: AX is negative (sign bit 1), so DX becomes FFFFH to preserve sign in DX:AX.

Tips

Exam rule: Before IDIV *r/m16*, use CWD to prepare a sign-extended dividend in DX:AX.

Quick Revision (One Look)

- **INC/DEC:** ± 1 (CF unaffected).
- **ADD/SUB:** arithmetic (no memory-to-memory).
- **ADC/SBB:** multi-precision add/sub using CF as carry/borrow.
- **MUL/IMUL:** AL/AX implicit; product in AX or DX:AX.
- **DIV/IDIV:** AX or DX:AX dividend; quotient/remainder in AL/AH or AX/DX.
- **NEG:** two's complement ($0 - \text{operand}$).
- **AAA/DAA/AAS/DAS:** adjust after BCD/ASCII add/sub (use immediately after ADD/-SUB).